

John Cheek
Alexander Card
CSC 481
8 September 2025

Milestone 1 Writeup

As with any coding project, the hardest part is always getting started. After going through the painstaking process of setting up SDL3 and SDL image we met as a team to discuss how we wanted to handle dividing the tasks. The initial engine setup was mostly provided in the boilerplate code. We decided that Benjamin would handle the input system, Judy would handle the collision detection system, I would implement the physics system, and since it's the biggest task, we all would work together on the entity system. I would create the generic entity type and the entity manager, Judy would work on the moving entities since they need to utilize the collision system, and Benjamin would work on the controllable entity since they are controlled via the input system. Since all of our team members are in CSC 481, there was no need for the scaling system.

The core graphics setup was relatively straightforward, as most of it was provided in the boilerplate code. As a team, we agreed upon that as the blueprint for our engine design. A while loop acts as the main game loop which runs until the player exits (or in the case of my individual game, a win condition is met). We also decided that it would be best to run all of our engine and game processes out of the main loop. In order to do that we figured that each system will need its own update function to repeatedly be called by the main game loop.

To implement the entity system, we knew our engine needed to support three types of entities: static entities, moving entities, and controllable entities. To manage this, our team decided we will split work on this up according to the other tasks we were working on but we first needed to make the parent entity class that contained basic information about the entity. As a team we decided that regardless of their specific functions, anything considered an entity in the game would have: a sprite, the sprite's size/entity's bounds, a position in the game window, and a flag to allow users to enable physics on the entity or not. Later, we decided that the entity should also have a name, which would make it easily accessible by other engine systems. We also chose to manage this information inside an Entity struct, as we figured that would be easier for memory management later down the line. Then we could make the moving and controllable entities subclasses of this generic entity type, with room to add more entity types with relative ease. While I was implementing the basic entity class, I realized it may make sense to include a sort of Entity Manager to easily retrieve, add, and delete entities in the game. I figured this entity manager could be managed as a singleton so it could be referenced easily by other entity systems. I ran this idea by the team, and they agreed, so Entity Manager was implemented as a singleton with functions to add, remove, and fetch entities from the vector array of all entities the user has created.

For the physics system we thought it best to implement it as its own class with an update function that applies physics to all entities with physics enabled. As a team, we figured the most straightforward way to implement physics is to initialize a default gravity value in the physics class but include a function to change it if the developer wishes for a different gravity. Since all the code for the physics system is based on the gravity value, changing it changes how physics enabled entities behave. In addition to the physics system class, we realized we would also need a vertical velocity variable in the generic entity struct, the physics class will then use this value to

change the entity's vertical position. This vertical velocity gets reset when the entity collides with a static object, or with the bottom of the screen.

The input system was perhaps the most straightforward to design. The input header has an array of boolean variables that correspond directly to keys on a keyboard and uses the SDL library's `SDL_GetKeyboardState`. By implementing it in this way, it's simple to check which keys are pressed by referencing the keyboard state. The keyboard state is constantly updating in the input classes' update function, which is called in the main game loop. Additionally, as we eventually hope to eventually make this into a specialized engine that focuses on the creation of card games, we also decided a good way to start on that would be to enable click and drag movement. By using click and drag the idea is that the player will be able to click on different 'cards' and place them within the game. Click and drag movement is enabled similarly to the keyboard state using the SDL Library's `SDL_GetMouseState` which gets the X and Y position of the mouse.

While rather straightforward on paper, the collision detection system was perhaps the most difficult part of this milestone (aside from installing the SDL3 and SDL_Image libraries). Unlike the other tasks, we figured the collision detection system would work best as part of the entity class, since all entities may benefit from using collisions. These could simply use the `SDL_Rect` functions to get the bounds of each sprite. Then, using the `SDL_HasRectIntersections(Entity a, Entity b)` if a collision is detected, it will log it to the console and return a boolean in our collisionDetection function. Having implemented it this way, all entities know when a collision occurs, but it is up to the game designer how to handle that collision. However; the collision detection function is called in all entities update functions, so it is constantly running in the background which could lead to some performance issues in the future. The primary challenge came when making objects 'solid' (e.g. a static platform the player can walk on) so that the player can actually collide with them. While it was difficult to come up with the math to change the players position according to the position of the other entity (especially if the entity was moving), there was also another potential runtime problem if this was added to the entity manager. In order to check for static collisions, all entities marked as solid need to loop through the list of other entities and check for collisions so that they do not collide with each other, which would lead to $O(n^2)$ in every iteration of the game loop. To solve this issue, our team decided that in this iteration, we would only check for these types of collisions with the player and all objects that set 'isSolid' to true. This same logic can be applied to any two entities, but to avoid future issues, the engine currently only checks for these static collisions between the player character and solid entities.

Finally, my individual game project 'A Dark Night' showcases all the core features of the engine in this first milestone through a simple platforming level as the 2D platformer structure effectively displays all the tasks needed for this milestone. The player starts on a static platform in the bottom left, and then needs to traverse through a series of static platforms and platforms, moving both horizontally and vertically at different speeds, towards the goal in the top left. If the player falls (hits the bottom of the screen) it teleports back to the starting platform. The game renders a 1920x1080 window in an off-black color to emulate the night sky. The player, platforms, background stars, and end goal are all entities using the entity system with the player being a controllable entity, the platforms being a static entity, the moving platforms being an auto moving entity, and the background stars and goal portal being generic animated entities. The player character showcases both the input system and the physics system. The player is controllable using A, W, and D, as well as click and drag movement (for debugging and

showcase purposes). The player also has physics applied so, after a jump, it will automatically fall down until it collides with a platform or the bottom of the screen. In order to demonstrate the adjustable gravity (and make the game a bit more forgiving) the game uses a value different from the default gravity value. The collision system is utilized throughout the game. If the player hits a platform, their vertical movement will stop. The player can also not pass through the platform from any direction. If the platform is moving, it will also update the player's position relative to the platform. The stars and the ending goal also demonstrate the flexibility of the entity system. The background stars are simply animated sprites to add detail to the scene with no collisions enabled. The goal portal is also an animated sprite but triggers the game win condition if the player interacts with it.